Bases de datos relacionales

None

Begoña Paterna

None

Índice

1. Guía de uso 3
2. Contenido 4
2.1 Introducción 4
2.2 Conexión a un SGBD 6
2.3 Operaciones sobre la BD 10
3. Material adicional 16
3.1 DB Browser for SQLite 16
3.2 DBeaver 20

1. Guía de uso

Estos apuntes están diseñados para que aprendas haciendo. A lo largo de la unidad, no solo veremos la teoría, sino que la aplicaremos directamente para construir, paso a paso, una aplicación completa de gestión de datos. El tema de la aplicación lo eliges tú, pero los pasos que daremos serán los mismos para todos.

Siguiendo la unidad no solo habrás aprendido los conceptos, sino que tendrás una aplicación completa y funcional creada por ti.

Intercaladas con la teoría y con los ejemplos encontrarás tres tipos de cajas. A continuación se explica que es cada una de ellas:

Ejecutar y analizar

Estas cajas son para analizar y comprender en detalle el ejemplo de código proporcionado. Tu tarea es ejecutar ese código, observar la salida y asegurarte de entender cómo y por qué funciona.

Práctica: Aplicar y construir

Estas cajas son prácticas que debes realizar tú. Es el momento de ponerte a programar y aplicar lo que acabas de aprender. Son los objetivos que debes completar para avanzar. Cada una de estas prácticas es un bloque que debes programar para ir avanzando en tu proyecto final. En cada práctica ampliarás lo de las anteriores.

Entrega

Estas cajas son entregas de tu trabajo. Las entregas pueden ser parciales (la profesora te dará sugerencias de mejora) o finales (la profesora calificará el trabajo que has realizado). No todas las prácticas llevan asociada una entrega.

2. Contenido

2.1 Introducción

Las bases de datos relacionales son esenciales en el desarrollo de aplicaciones modernas. Su integración con una aplicación requiere realizar una **conexión** al sistema gestor de base de datos (SGBD) desde el lenguaje de programación. Este tema se centra en cómo realizar esa conexión, cómo trabajar con datos mediante sentencias SQL y cómo aplicar buenas prácticas, como el cierre de recursos, el uso de transacciones y procedimientos almacenados.

Una **base de datos relacional** es un sistema de almacenamiento de información que **organiza los datos en tablas**. Cada tabla representa una entidad (por ejemplo, clientes, productos, facturas) y está compuestas por filas y columnas, donde cada fila representa un registro único y cada columna contiene un atributo específico de ese registro. Estas bases de datos (BD) siguen el **Modelo Relacional**, desarrollado por Edgar F. Codd en la década de 1970, y permite establecer vínculos o **relaciones entre diferentes tablas** mediante **claves primarias y foráneas**, facilitando así la integridad, la coherencia y la eficiencia en el manejo de grandes volúmenes de datos.

Ejemplo de tabla `clientes`:

id_cliente	nombre	ciudad
1	Pol	Castellón
2	Eli	Valencia

La **Clave primaria (Primary Key)** es una columna (o conjunto de columnas) que **identifica de forma única** cada fila de una tabla. En el ejemplo: `id_cliente` es clave primaria en la tabla `clientes`.

La **Clave foránea (Foreign Key)** es una columna que **hace referencia a una clave primaria de otra tabla** para establecer una relación. Por ejemplo, en otra tabla llamada `facturas`, el campo `id_cliente` puede ser clave foránea que apunta al campo id_clientes de la tabla `clientes`.

Ejemplo de tabla `facturas`:

id_factura	id_cliente	fecha
1	1	2025-09-18
2	1	2025-09-18

El lenguaje **SQL (Structured Query Language)** se utiliza para gestionar bases de datos relacionales ya que gracias a él se pueden crear, modificar, consultar y eliminar datos de forma sencilla y estandarizada, lo que lo convierte en la opción preferida para una amplia variedad de aplicaciones empresariales y tecnológicas. Algunos de sus comandos básicos son:

- `SELECT`: consultar datos
- `INSERT`: añadir registros
- `UPDATE`: modificar datos existentes
- `DELETE`: eliminar registros
- `CREATE`: definir tablas, claves, relaciones, etc.

Es lo que se denomina **CRUD**, es decir, **C**reate (crear), **R**ead (Leer), **U**pdate (Actualizar) y **D**elete (Borrar).

Un ejemplo sencillo de consulta podría ser:

```
SELECT nombre FROM clientes WHERE ciudad = 'Valencia';
```

Tipos de SGBD relacionales

Conocer qué **tipo de gestor de base de datos** se está utilizando es esencial para poder **conectar** correctamente desde una aplicación, ya que cada uno necesita su propio conector o driver. Podemos encontrar:

1. Gestores independientes (cliente-servidor): PostgreSQL, MySQL, Oracle, SQL Server...

- Sistemas robustos y escalables, ideales para entornos multi-usuario y aplicaciones web.
- Requieren un servidor dedicado y una configuración más compleja.
- Casos de uso: aplicaciones web, servicios empresariales, sistemas con alta demanda de acceso concurrente.

2. Gestores embebidos: SQLite, H2, Derby...

- Base de datos ligera, sin servidor, ideal para aplicaciones móviles o de escritorio donde no se requiere gestión centralizada.
- Fácil de configurar y desplegar, ya que la base de datos reside en un archivo local.
- Casos de uso: aplicaciones de escritorio, móviles, prototipos, pruebas unitarias.

Práctica 1: Crea tu proyecto y tu base de datos

1. Crea un nuevo proyecto en Kotlin con Gradle.
2. A partir del fichero de información utilizado en el proyecto de la unidad anterior, crea una base de datos SQLite **nombre_de_tu_BD.sqlite** con una tabla que contenga la información del fichero.
3. Crea una carpeta **datos** dentro de tu proyecto y copia en ella el archivo **nombre_de_tu_BD.sqlite**.

2.2 Conexión a un SGBD

Cuando desarrollamos aplicaciones que trabajan con información persistente, necesitamos acceder a BD para consultar, insertar, modificar o eliminar datos. Existen dos formas principales de hacerlo desde el código:

- Acceso mediante ORM (Object-Relational Mapping).
- Acceso mediante conectores.

Acceso mediante ORM

Un **ORM** es una herramienta que permite trabajar con la base de datos como si fuera un conjunto de objetos, evitando tener que escribir directamente SQL. El **ORM** se encarga de mapear las tablas a clases y los registros a objetos, y traduce automáticamente las operaciones del código a consultas SQL. Es ideal para trabajar de forma más productiva en aplicaciones complejas. Sus principales características son:

- Se trabaja con clases en lugar de tablas SQL.
- Ahorra mucho código repetitivo.
- Ideal para proyectos medianos o grandes que requieren mantener muchas entidades.

Algunos ejemplos de ORMs

ORM / Framework	Lenguaje	Descripción
Hibernate	Java/Kotlin	El ORM más utilizado con JPA
Exposed	Kotlin	ORM ligero y expresivo creado por JetBrains
Spring Data JPA	Java/Kotlin	Abstracción que automatiza el acceso a datos
Room	Java/Kotlin	ORM oficial para bases de datos SQLite en Android

JPA (Java Persistence API) es una especificación estándar de Java que define cómo se deben mapear objetos Java (o Kotlin) a tablas de bases de datos relacionales. Es decir, permite gestionar la persistencia de datos de forma orientada a objetos, sin necesidad de escribir SQL directamente. Es el estándar utilizado por las herramientas ORM como Hibernate, EclipseLink, o Spring Data JPA.

Acceso mediante conectores

Un **conector** (también llamado driver) es una librería software que permite que una aplicación se comunique con un gestor de base de datos (SGBD). Actúa como un puente entre nuestro código y la base de datos, traduciendo las instrucciones SQL a un lenguaje que el gestor puede entender y viceversa. Sin un conector, tu aplicación no podría comunicarse con la base de datos.

Una base de datos puede ser accedida desde diferentes orígenes o herramientas, siempre que tengamos:

- Las credenciales de acceso (usuario y contraseña)
- El host/servidor donde se encuentra la base de datos
- El motor de base de datos (PostgreSQL, MySQL, SQLite, etc.)
- Los puertos habilitados y los permisos correctos

Las principales formas de conectarse a una base de datos son las siguientes:

Medio de conexión	Descripción
Aplicaciones de escritorio	Herramientas gráficas como **DBeaver**, **pgAdmin**, **MySQL Workbench**, **DB Browser for SQLite**. Permiten explorar, consultar y administrar BD de forma visual.
Aplicaciones desarrolladas en código	Programas en **Kotlin**, **Java**, **Python**, **C#**, etc., mediante **conectores** como **JDBC**, **psycopg2**, **ODBC**, etc. para acceder a BD desde código.
Línea de comandos	Clientes como `psql` (PostgreSQL), `mysql`, `sqlite3`. Permiten ejecutar comandos SQL directamente desde terminal.
Aplicaciones web	Sitios web que acceden a BD desde el backend (por ejemplo, en Spring Boot, Node.js, Django, etc.).
APIs REST o servidores intermedios	Servicios web que conectan la BD con otras aplicaciones, actuando como puente o capa de seguridad.
Aplicaciones móviles	Apps Android/iOS que acceden a BD locales (como **SQLite**) o remotas (vía **Firebase**, API REST, etc.).
Herramientas de integración de datos	Software como **Talend**, **Pentaho**, **Apache Nifi** para migrar, transformar o sincronizar datos entre sistemas.

De todas las formas posibles de interactuar con una base de datos, nos vamos a centrar en el uso de **conectores JDBC (Java Database Connectivity)**. **JDBC** es una API estándar de Java (y compatible con Kotlin) que permite conectarse a una BD, enviar instrucciones SQL y procesar los resultados manualmente. Es el método de más bajo nivel, pero ofrece un control total sobre lo que ocurre en la BD. Es ideal para aprender los fundamentos del acceso a datos y tener control total y aprenderlo ayuda a entender mejor lo que hace un ORM por debajo. Sus principales características son:

- El programador escribe directamente las consultas SQL.
- Requiere gestionar manualmente conexiones, sentencias y resultados.
- Se necesita un driver específico (conector) para cada SGBD:

Algunos ejemplos de conectores

SGBD	Conector (Driver JDBC)	URL de conexión típica
PostgreSQL	org.postgresql.Driver	jdbc:postgresql://host:puerto/basedatos
MySQL / MariaDB	com.mysql.cj.jdbc.Driver	jdbc:mysql://host:puerto/basedatos
SQLite (embebido)	org.sqlite.JDBC	jdbc:sqlite:ruta_al_fichero

Una aplicación (escrita en Kotlin, Java u otro lenguaje) puede leer, insertar o modificar información almacenada en una base de datos relacional si previamente se ha conectado al sitema gestor de base de datos (SGBD). **JDBC** (Java Database Connectivity) es la API básica de Java (conector) para conectarse a bases de datos relacionales y su sintaxis general es:

```
jdbc:<gestor>://<host>:<puerto>/<nombre_base_datos>
```

Aunque puede variar según el SGBD con el que se trabaje. Por ejemplo en SQLite no se necesita usuario ni contraseña ya que es una base de datos local y embebida.

SGBD	URL de conexión
PostgreSQL	jdbc:postgresql://localhost:5432/plantas
MySQL	jdbc:mysql://localhost:3306/plantas
SQLite	jdbc:sqlite:plantas.sqlite

También dependiendo del SGBD será necesario utilizar un **conector JDBC** u otro. Para ello utilizaremos la herramienta **Gradle**, que permite automatizar la gestión de dependencias sin tener que configurar nada a mano añadiendo las líneas correspondientes en el fichero **build.gradle.kts**. A continuación se muestran las líneas para los SGBD PostgreSQL, MySQL y SQLite.

```
dependencies {
    implementation("org.postgresql:postgresql:42.7.1") //Postgres
    implementation("mysql:mysql-connector-java:8.3.0") //MySQL
    implementation("org.xerial:sqlite-jdbc:3.43.0.0") //SQLite
}
```

Conexión a SQLite

A continuación se describe cómo conectar a una base de datos **SQLite** llamada **plantas.sqlite** que se encuentra en la carpeta **datos** dentro de un proyecto en **Kotlin**.

```
import java.io.File
import java.sql.DriverManager

fun main() {
    // Ruta al archivo de base de datos SQLite
    val dbPath = "src/main/resources/plantas.sqlite"
    val dbFile = File(dbPath)
    println("Ruta de la BD: ${dbFile.absolutePath}")
    val url = "jdbc:sqlite:${dbFile.absolutePath}"

    // Conexión y prueba
    DriverManager.getConnection(url).use { conn ->
        println("Conexión establecida correctamente con SQLite.")
    }
}
```

Realiza lo siguiente

1. Crea un proyecto kotlin con gradle o utiliza uno que ya tengas.
2. Añade las dependencias para trabajar con SQLite.
3. Descarga el fichero con la base de datos de ejemplo desde el siguiente enlace: plantas.sqlite
4. Copia el fichero en la carpeta correcta del proyecto.
5. Ejecuta el programa y comprueba que la salida por consola es la siguiente:

   Ruta de la BD: F:...\src\main\resources\plantas.sqlite

   Conexión establecida correctamente con SQLite

Práctica 2: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

Una buena páctica es cerrar la conexión a la base de datos después de realizar operaciones sobre ella para que permanezca abierta el mínimo tiempo posible. Una forma de conseguir esto es tener las funciones de conexión y desconexión en un objeto separado.

Además, otra ventaja de trabajar así es que si la base de datos cambia de ubicación, solo habrá que actualizar la ruta dentro del código del objeto que maneja la conexión y no en varios sitios del programa.

Un ejemplo del código podría ser el siguiente:

```
import java.io.File
import java.sql.Connection
import java.sql.DriverManager
import java.sql.SQLException

object BD {
    // Ruta al archivo de base de datos SQLite
    private val dbPath = "datos/plantas.sqlite"
    private val dbFile = File(dbPath)
    private val url = "jdbc:sqlite:${dbFile.absolutePath}"

    // Obtener conexión
```

```
    fun getConnection(): Connection? {
        return try {
            DriverManager.getConnection(url)
        } catch (e: SQLException) {
            e.printStackTrace()
            null
        }
    }

    // Función de prueba: verificar conexión
    fun testConnection(): Boolean {
        return getConnection()?.use { conn ->
            println("Conexión establecida con éxito a ${dbFile.absolutePath}")
            true
        } ?: false
    }

    // Cerrar conexión
    fun closeConnection(conn: Connection?) {
        try {
            conn?.close()
            println("Conexión cerrada correctamente.")
        } catch (e: SQLException) {
            println("Error al cerrar la conexión: ${e.message}")
        }
    }
}
```

De esta forma, cuando el programa necesite acceder a la BD llamará a la función de conexión y cuando termine llamará a la función que cierra la conexión. Un ejemplo de estas llamadas podría ser:

```
fun main() {
    val conn = BD.getConnection()
    if (conn != null) {
        println("Conectado a la BD correctamente.")
        BD.closeConnection(conn)
    }
}
```

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 3: Optimiza el manejo de la BD de tu aplicación

1. Crea el archivo con las funciones de conexión y desconexión a la BD.
2. Comprueba desde el main que el programa se conecta a la BD correctamente y luego cierra la conexión.

2.3 Operaciones sobre la BD

En **JDBC** (Java Database Connectivity), las operaciones sobre la base de datos se realizan utilizando los siguientes objetos y métodos:

- **Connection**, establece el canal de comunicación con el SGBD (PostgreSQL, MySQL, etc.)
- Los objetos **PreparedStatement** y **CreateStatement** se utlizan para enviar consultas SQL desde el programa a la base de datos. A continuación se muestra una tabla con el uso de cada uno:

Si necesitas...	Usa...
Consultas sin parámetros	`CreateStatement`
Consultas con datos del usuario	`PreparedStatement`
Seguridad frente a inyecciones SQL	`PreparedStatement`
Ejecutar muchas veces con distintos valores	`PreparedStatement`
Crear tablas o sentencias SQL complejas que no cambian	`CreateStatement`

- Los métodos **executeQuery()**, **executeUpdate()** y **execute()** se utilizan para ejecutar sentencias SQL, pero se usan en contextos diferentes. A continuación se muestra una tabla con el uso de cada uno:

Método	Uso principal	Tipo de sentencia SQL	Resultado que devuelve
executeQuery()	Realizar consultas	SELECT	Objeto **ResultSet** con el resultado de la consulta SQL. Permite recorrer fila a fila el conjunto de resultados, accediendo a cada campo por nombre o por posición
executeUpdate()	Realizar modificaciones	INSERT, UPDATE, DELETE, DDL (CREATE, DROP, etc.)	Entero con el número de filas afectadas
execute()	No se sabe de antemano qué tipo de sentencia SQL se va a ejecutar (consulta o modificación)	Sentencias SQL que pueden devolver varios resultados	Booleano **true** si el resultado es un ResultSet (SELECT) y **false** si el resultado es un entero (INSERT, UPDATE, DELETE,CREATE, ALTER)

Liberación de recursos

Cuando una aplicación accede a una base de datos, abre varios recursos internos que consumen memoria y conexiones activas en el sistema:

- La conexión con el servidor de base de datos (Connection).
- Las sentencias SQL preparadas (Statement o PreparedStatement).
- El resultado de la consulta (ResultSet).

Estos recursos no se liberan automáticamente cuando se termina su uso (especialmente en Java o Kotlin con JDBC). Si no se cierran correctamente, se pueden producir problemas como:

- Fugas de memoria.
- Bloqueo de conexiones (demasiadas conexiones abiertas).
- Degradación del rendimiento.
- Errores inesperados en la aplicación.

En Kotlin, puedes usar **use {}** para cerrar recursos automáticamente al finalizar el bloque.

Si no utilizas **use {}** en Kotlin (o try-with-resources en Java), entonces debes cerrar manualmente cada uno de los recursos abiertos (ResultSet, Statement y Connection) usando **.close()**, y normalmente deberías hacerlo dentro de un bloque **finally** para garantizar su cierre incluso si ocurre un error. El orden correcto de cierre es del más interno al más externo.

Ejemplos en SQlite

Los siguientes ejemplos se han realizado sobre la tabla `plantas` de la BD `plantas.sqlite` y se ha utilizado las funciones de conexión y desconexión programadas anteriormente en un objeto en un archivo **.kt** separado

Ejemplo 1 - Consulta sin parámetros: El siguiente ejemplo consulta toda la infromación de las plantas y la mustra por consola de forma ordenada:

```
fun consultarPlantas() {
    val conn = BD.getConnection()
    if (conn != null) {
        val sql = "SELECT * FROM plantas"
        conn.prepareStatement(sql).use { stmt ->

            stmt.executeQuery().use { rs ->
                println("Plantas encontradas:")

                while (rs.next()) {
                    val idPlanta = rs.getString("id")
                    val nombreComun = rs.getString("nombre_comun")
                    val nombreCientifico = rs.getString("nombre_cientifico")
                    val frecuenciaRiego = rs.getInt("frecuencia_riego")
                    val altura = rs.getDouble("altura")

                    println("- $idPlanta: $nombreComun ($nombreCientifico). Frecuencia de riego: $frecuenciaRiego, altura: $altura")
                }
            }
        }
        BD.closeConnection(conn)
    }
}
```

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 4: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

Ejemplo 2 - Consulta con parámetros: El siguiente ejemplo consulta la infromación de la planta cuyo ID coincide con uno proporcionado por el usuario:

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 5: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

Ejemplo 3 - INSERT: El siguiente ejemplo añade un registro a la tabla de la BD.

```
fun insertarPlanta() {
    val conn = BD.getConnection()
    if (conn != null) {
        //datos a insertar
        val nombreComun = "Palmera"
        val nombreCientifico = "Arecaceae"
        val frecuenciaRiego = 2
        val altura = 8.5

        val sql = "INSERT INTO plantas (nombre_comun, nombre_cientifico, frecuencia_riego, altura) VALUES (?, ?, ?, ?)"

        conn.prepareStatement(sql).use { stmt ->
            stmt.setString(1, nombreComun)
            stmt.setString(2, nombreCientifico)
            stmt.setInt(3, frecuenciaRiego)
            stmt.setDouble(4, altura)
            stmt.executeUpdate()
        }
        BD.closeConnection(conn)
        println("Planta insertada correctamente")
    }
}
```

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 6: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

Ejemplo 4 - UPDATE: El siguiente ejemplo actualiza la `altura` de una planta identificada por su `id` (el id de la planta a actualizar y la nueva altura se piden por consola).

```
fun actualizarPlanta() {
    var idPlanta = 0
    var nuevaAltura = 0.0

    print("Introduce un ID: ")
    val pideID = readLine()
    if (pideID != null && pideID.isNotBlank()) {
        idPlanta = pideID.toInt()

        print("Introduce la nueva altura: ")
        val pideAltura = readLine()
        if (pideAltura != null && pideAltura.isNotBlank()) {
            nuevaAltura = pideAltura.toDouble()
        }
    }

    val conn = BD.getConnection()
    if (conn != null) {
        val sql = "UPDATE plantas SET altura = ? WHERE id = ?"

        conn.prepareStatement(sql).use { stmt ->
            stmt.setDouble(1, nuevaAltura)
            stmt.setInt(2, idPlanta)
            stmt.executeUpdate()
        }
        BD.closeConnection(conn)
        println("Planta actualizada correctamente")
    }
}
```

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 7: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

Ejemplo 5 - DELETE: El siguiente ejemplo elimina una planta identificada por su `id` que se pide por consola.

```
fun eliminarPlanta() {
    print("Introduce un ID: ")
    val pideID = readLine()
    if (pideID != null && pideID.isNotBlank()) {
        val idPlanta = pideID.toInt()

        val conn = BD.getConnection()
        if (conn != null) {
            val sql = "DELETE FROM plantas WHERE id = ?"
            conn.prepareStatement(sql).use { stmt ->
                stmt.setInt(1, idPlanta)
                stmt.executeUpdate()
            }
            BD.closeConnection(conn)
            println("Planta eliminada correctamente")
        }
    }
}
```

Realiza lo siguiente

Prueba el código de ejemplo y verifica que funciona correctemente.

Práctica 8: Conecta a tu base de datos

Replica el ejemplo anterior para que funcione con tu base de datos.

FALTA a PARTIR DE AQUÍ

Consultas complejas: JOIN, filtros y ordenaciones

Ejemplo_join.kt: Este ejemplo obtiene las líneas de factura con nombre del artículo y ordenado por numero de factura y línea.

```
    package SQLite
    import java.sql.DriverManager

    fun main() {
        val dbPath = "src/main/resources/Tienda.sqlite"
        val dbFile = java.io.File(dbPath)
        val url = "jdbc:sqlite:${dbFile.absolutePath}"

        DriverManager.getConnection(url).use { conn ->

            val sql = """
                SELECT lf.num_f, lf.num_l, lf.cod_a, a.descrip, lf.quant, lf.preu
                FROM linia_fac lf
                JOIN article a ON lf.cod_a = a.cod_a
                ORDER BY lf.num_f, lf.num_l
            """.trimIndent()

            conn.prepareStatement(sql).use { stmt ->
                stmt.executeQuery().use { rs ->
                    println("Líneas de factura:")
                    println("Factura | Línea | Artículo | Descripción | Cantidad | Precio")

                    while (rs.next()) {
                        val numF = rs.getInt("num_f")
                        val numL = rs.getInt("num_l")
                        val codA = rs.getString("cod_a")
                        val descrip = rs.getString("descrip")
                        val quant = rs.getInt("quant")
                        val preu = rs.getDouble("preu")
```

```
                    println("$numF\t$numL\t$codA\t$descrip\t$quant\t$preu")
                }
            }
        }
    }
}
```

Ejemplos para cerrar recursos abiertos sin **use()**, de forma manual y con el bloque **try-catch-finally**

Ejemplo_cierre_manual.kt: Cierra los recurso con close()

```
package SQLite
import java.sql.DriverManager
import java.sql.Connection
import java.sql.PreparedStatement
import java.sql.ResultSet

fun main() {
    val dbPath = "src/main/resources/Tienda.sqlite"
    val dbFile = java.io.File(dbPath)
    val url = "jdbc:sqlite:${dbFile.absolutePath}"

    val conn: Connection = DriverManager.getConnection(url)
    val sql = "SELECT cod_a, descrip, preu, stock, stock_min FROM article"
    val stmt: PreparedStatement = conn.prepareStatement(sql)
    val rs: ResultSet = stmt.executeQuery()

    println("Artículos:")
    println("Código\tDescripción\tPrecio\tStock\tStock mín.")

    while (rs.next()) {
        val codA = rs.getString("cod_a")
        val descrip = rs.getString("descrip")
        val preu = rs.getDouble("preu")
        val stock = rs.getInt("stock")
        val stockMin = rs.getInt("stock_min")

        println("$codA\t$descrip\t$preu\t$stock\t$stockMin")
    }

    rs.close()
    stmt.close()
    conn.close()
}
```

Ejemplo_cierre_try_catch.kt: Cierra los reursos con try-catch-finally

```
package SQLite
import java.sql.Connection
import java.sql.DriverManager
import java.sql.PreparedStatement
import java.sql.ResultSet

fun main() {
    val dbPath = "src/main/resources/Tienda.sqlite"
    val dbFile = java.io.File(dbPath)
    val url = "jdbc:sqlite:${dbFile.absolutePath}"

    var conn: Connection? = null
    var stmt: PreparedStatement? = null
    var rs: ResultSet? = null

    try {
        conn = DriverManager.getConnection(url)
        val sql = "SELECT cod_a, descrip, preu, stock, stock_min FROM article"
        stmt = conn.prepareStatement(sql)
        rs = stmt.executeQuery()

        println("Artículos:")
        println("Código\tDescripción\tPrecio\tStock\tStock mín.")

        while (rs.next()) {
            val codA = rs.getString("cod_a")
            val descrip = rs.getString("descrip")
            val preu = rs.getDouble("preu")
            val stock = rs.getInt("stock")
            val stockMin = rs.getInt("stock_min")

            println("$codA\t$descrip\t$preu\t$stock\t$stockMin")
        }

    } catch (e: Exception) {
        println("Error al acceder a la base de datos: ${e.message}")
    } finally {
        try { rs?.close() } catch (e: Exception) { /* Ignorar */ }
        try { stmt?.close() } catch (e: Exception) { /* Ignorar */ }
        try { conn?.close() } catch (e: Exception) { /* Ignorar */ }
```

```
        }
    }
```

3. Material adicional

3.1 DB Browser for SQLite

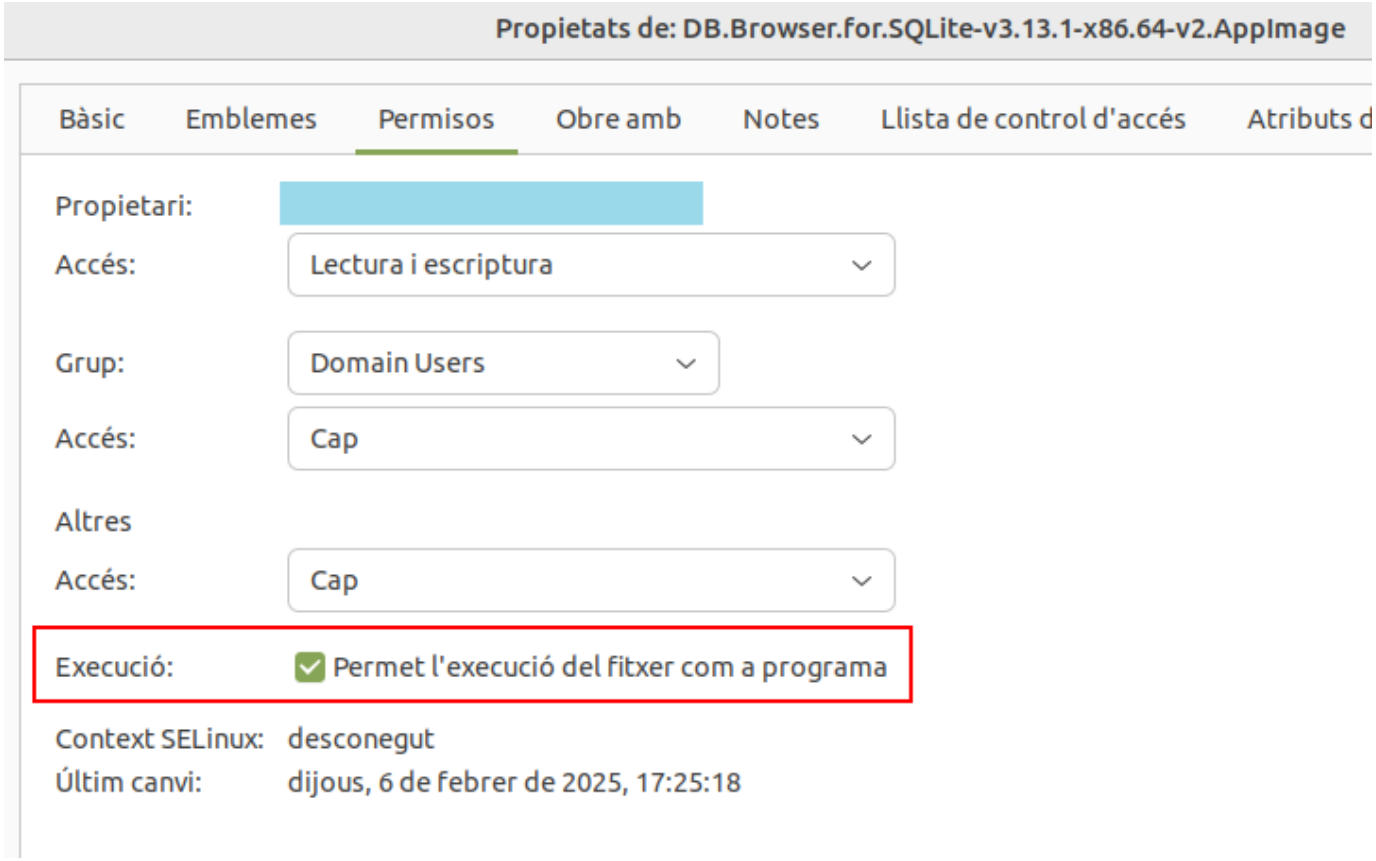
Siempre que desarrollamos aplicaciones que acceden a una base de datos es recomendable disponer de un visor / editor adecuado al sistema gestor de bases de datos con el que estamos trabajando para poder realizar comprobaciones. Con la herramienta DB Browser for SQLite podemos crear, modificar y consultar la estructura de una base de datos.

Se puede descargar desde el siguiente enlace https://sqlitebrowser.org/dl/

Para Linux puedes descargar la AppImage que no necesita instalación desde: https://sqlitebrowser.org/dl/#linux

Una vez descargada, para poder ejecutarla con doble clic, sigue estos pasos:

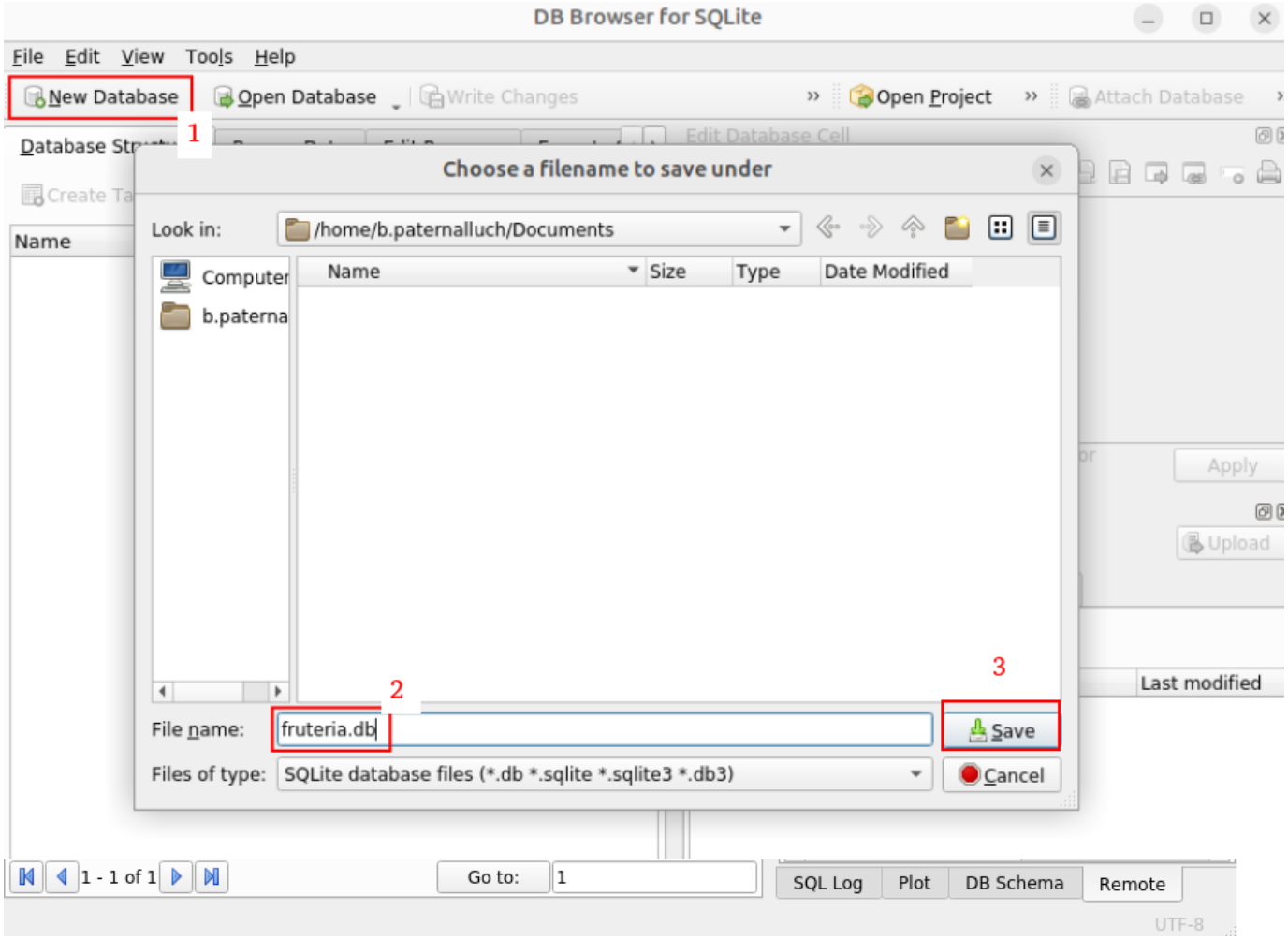
- Haz clic con el botón derecho y entra en Propiedades.
- Entra en la pestaña Permisos.
- Marca la casilla Permite la ejecución del fichero como programa.



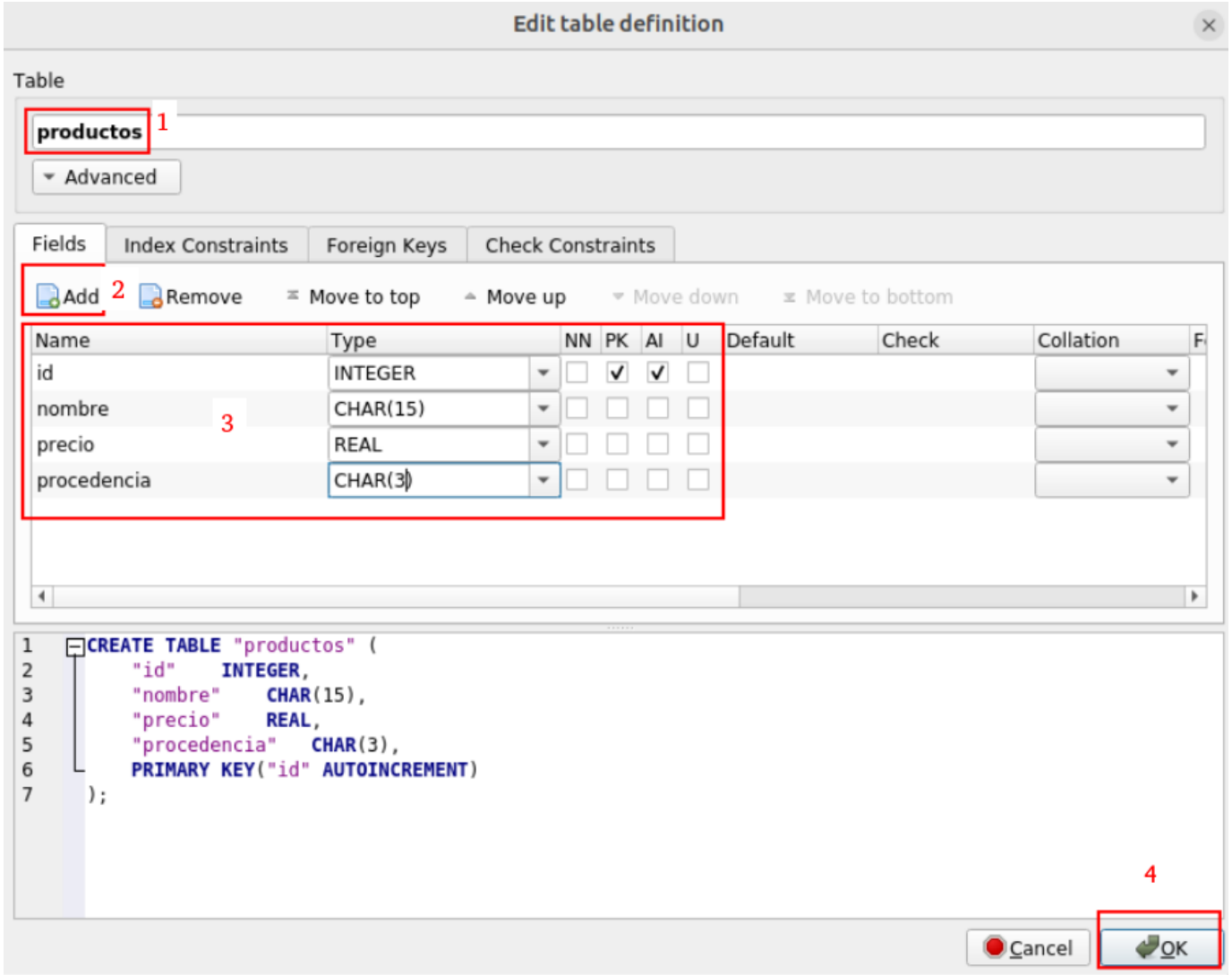
A continuación se describen los pasos a seguir para crear una base de datos llamada **fruteria.db** con una tabla llamada **productos** con la siguiente estructura:

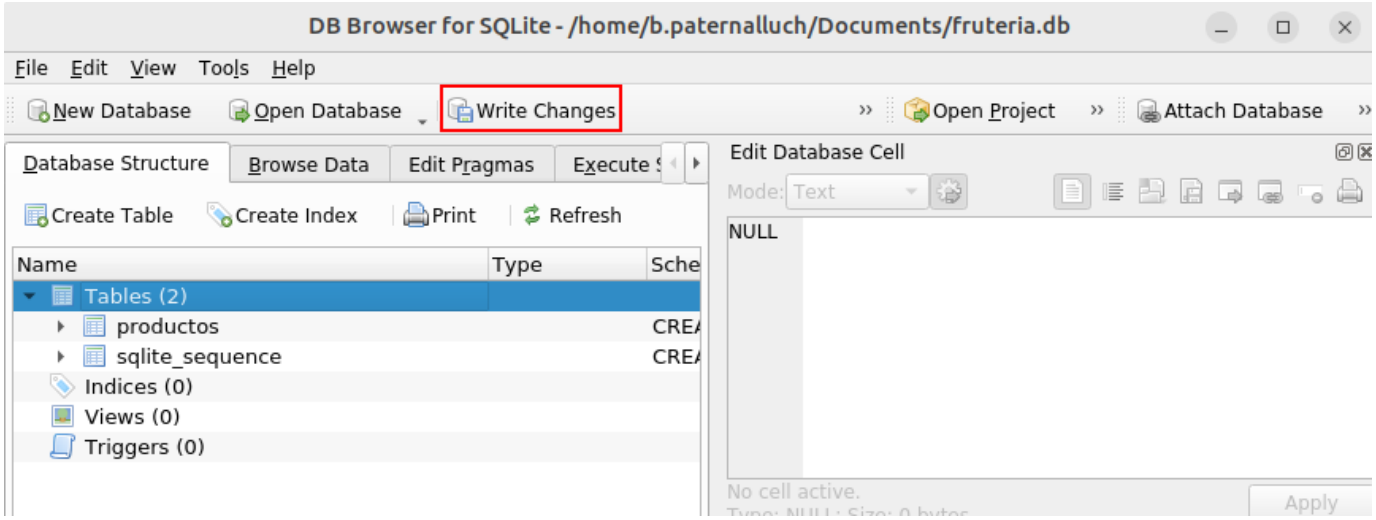
Campo	Tipo de Dato	Descripción
id	INTEGER (PK, AUTOINCREMENT)	Identificador único.
nombre	CHAR(15)	Nombre.
precio	REAL	Precio (con decimales).
procedencia	CHAR(3)	Código de país de procedencia.

Crear la base de datos

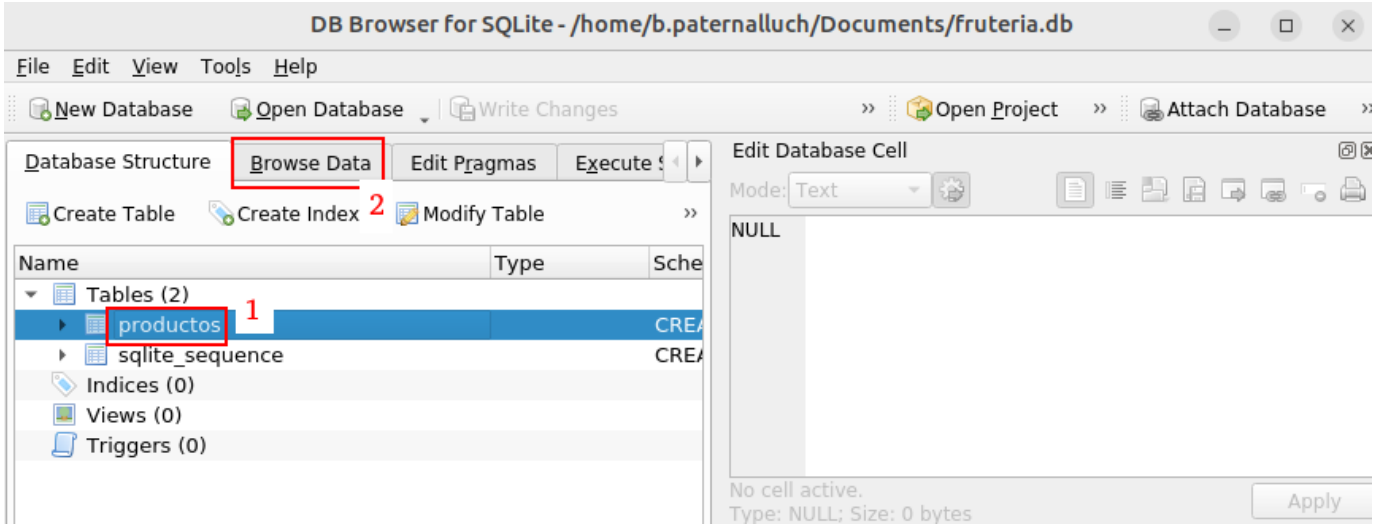


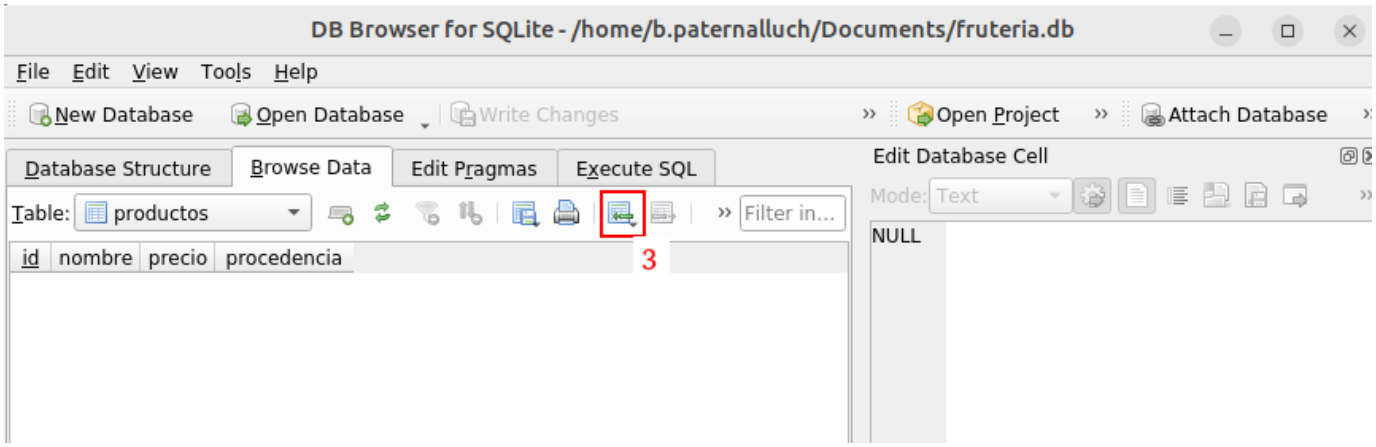
Crear la tabla y los campos

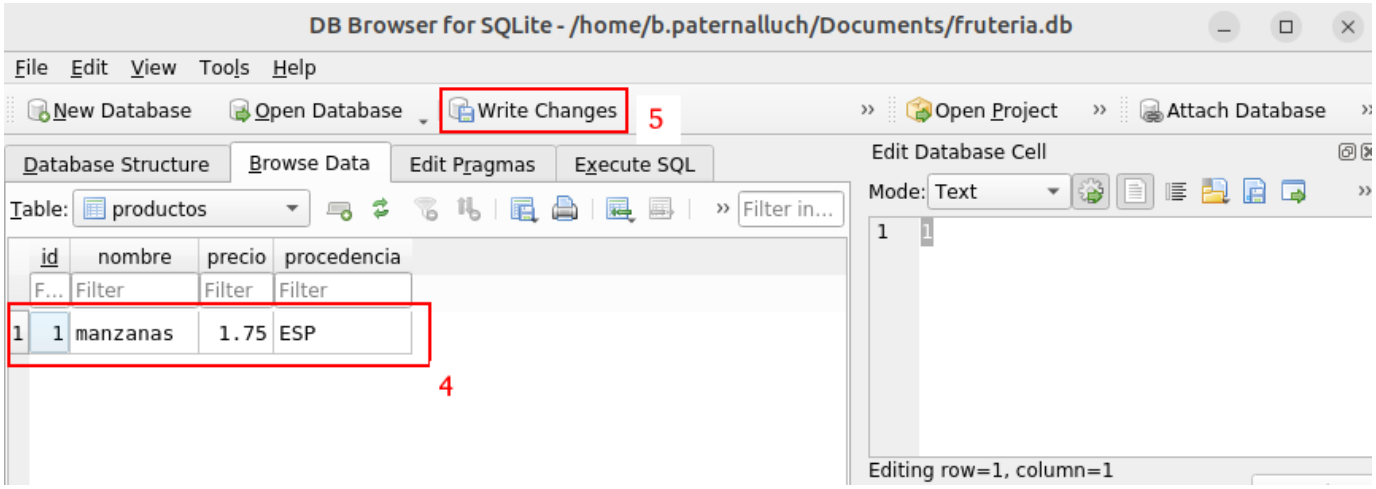




Añadir información a la tabla







3.2 DBeaver

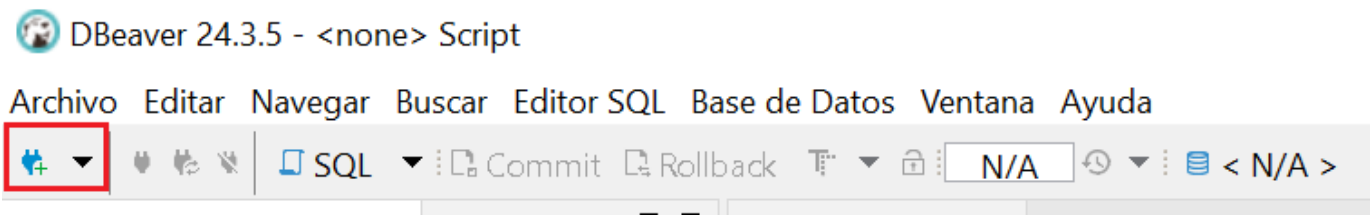
DBeaver es una herramienta gráfica y gratuita que permite gestionar múltiples bases de datos de forma visual. Algunas de las acciones que podemos realizar con esta herramienta son las siguientes:

- Explorar la estructura de la base de datos (tablas, vistas, claves, relaciones...).
- Consultar datos.
- Modificar tablas, añadir registros o ejecutar scripts SQL sin salir del proyecto.
- Probar consultas antes de implementarlas en el programa.

Los siguientes pasos ilustran como conectar a la BD de ejemplo **plantas.db** que ya tenemos copiada a la carpeta **resources** de nuestro proyecto.

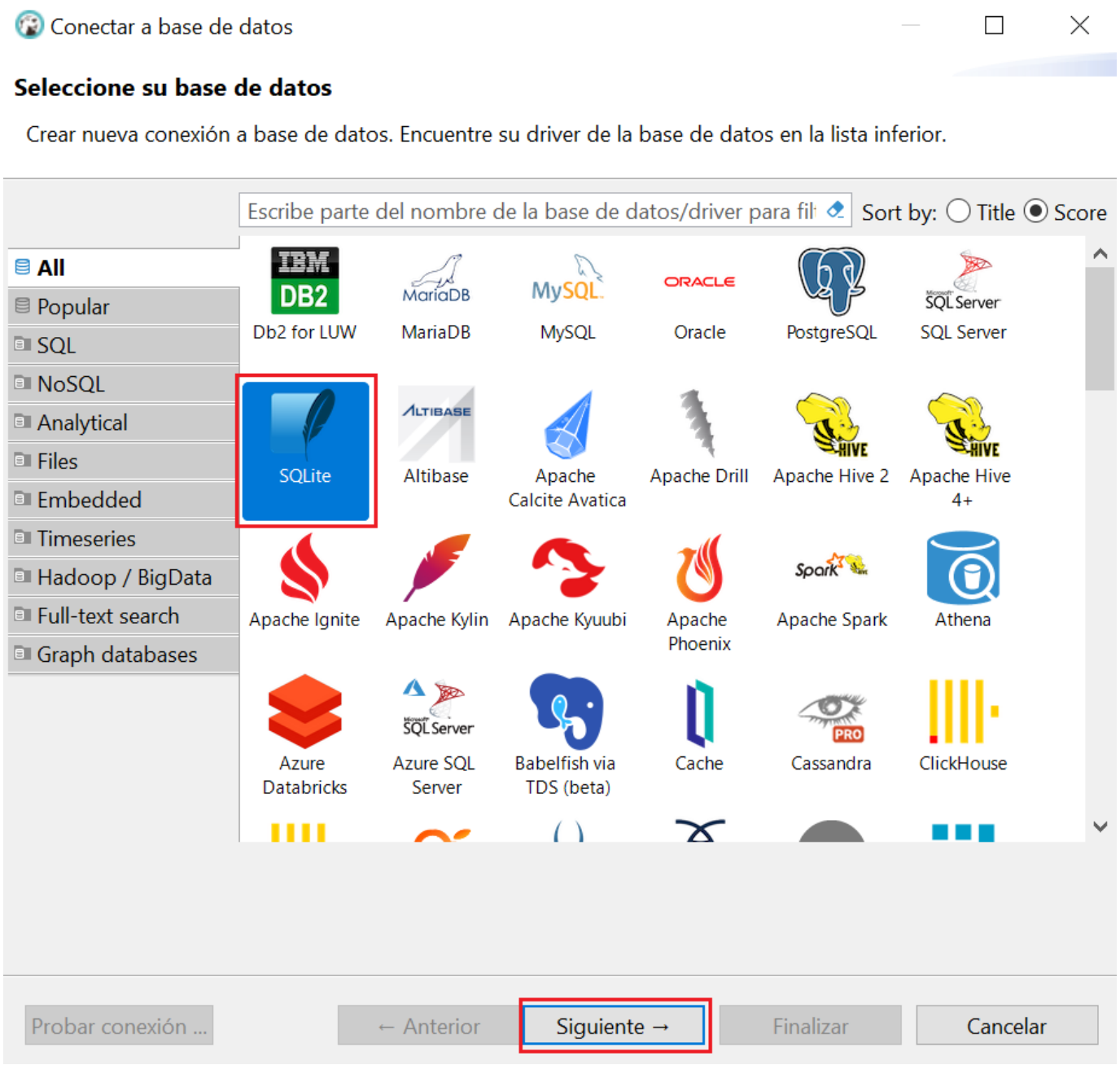
1. Abre DBeaver

Inicia el programa y haz clic en el botón **"Nueva conexión"** (ícono de enchufe) o ve al menú `Archivo > Nueva conexión`.

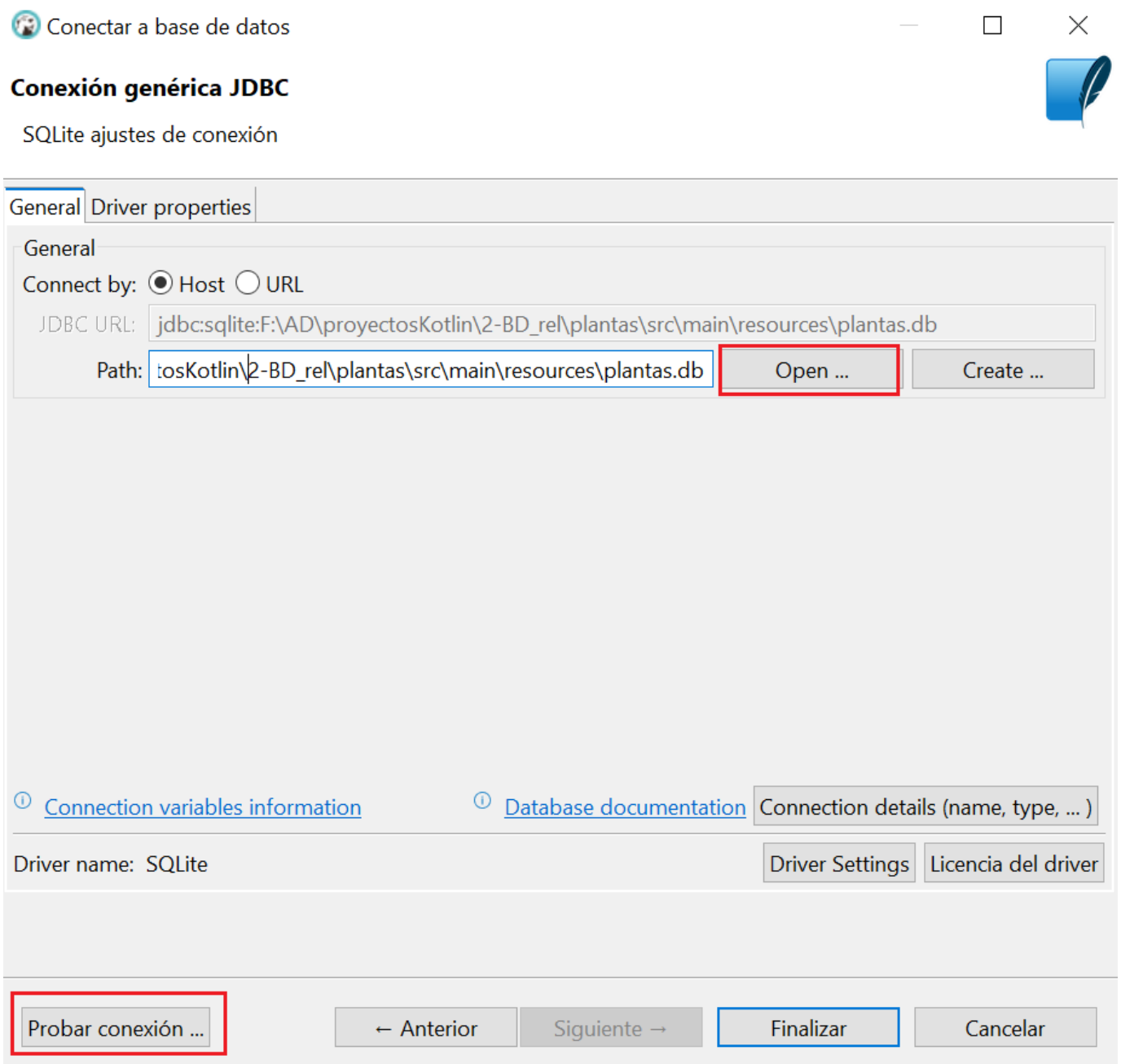


2. Selecciona el tipo de base de datos

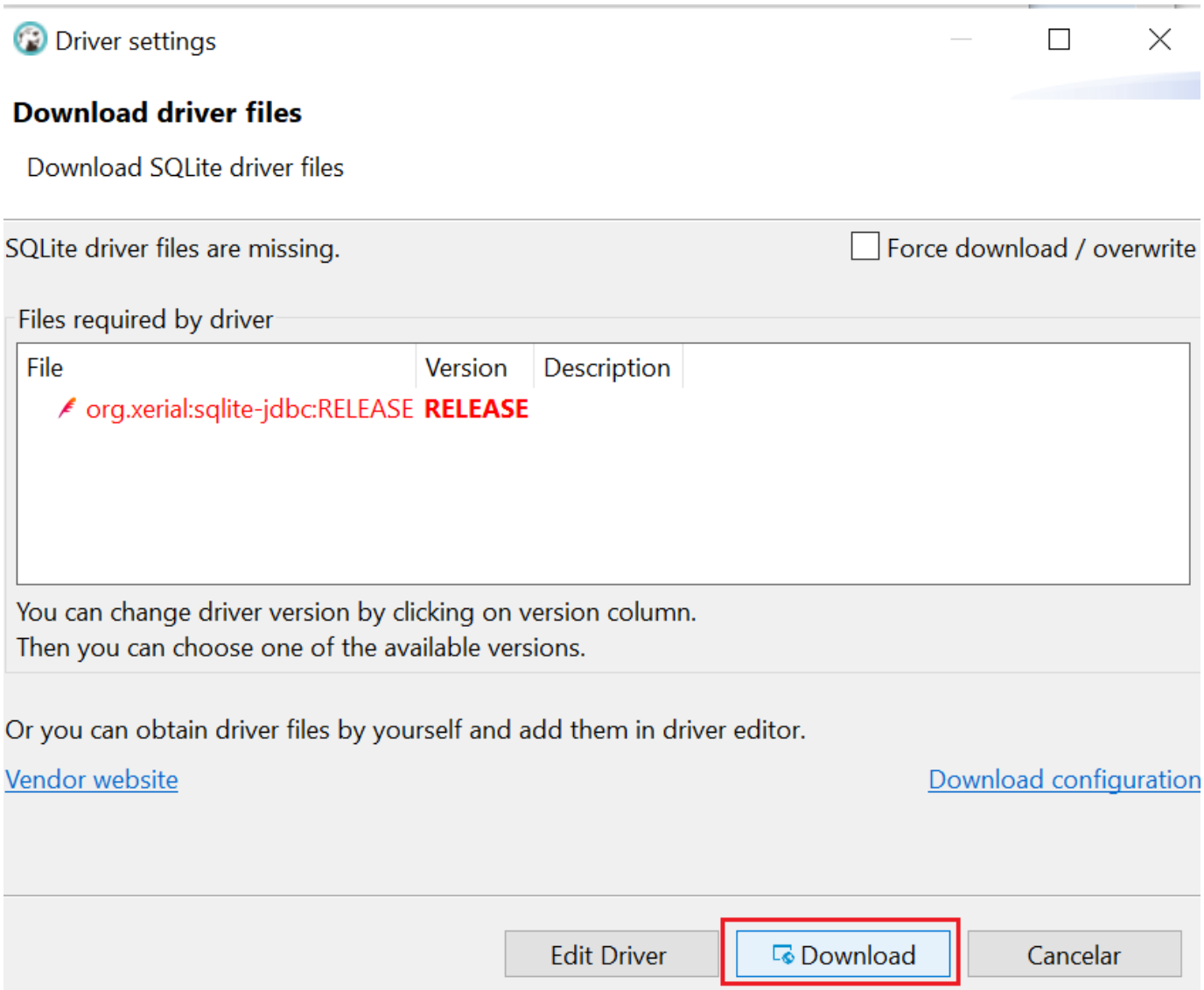
En la ventana de selección, elige **SQlite** y pulsa **Siguiente**.



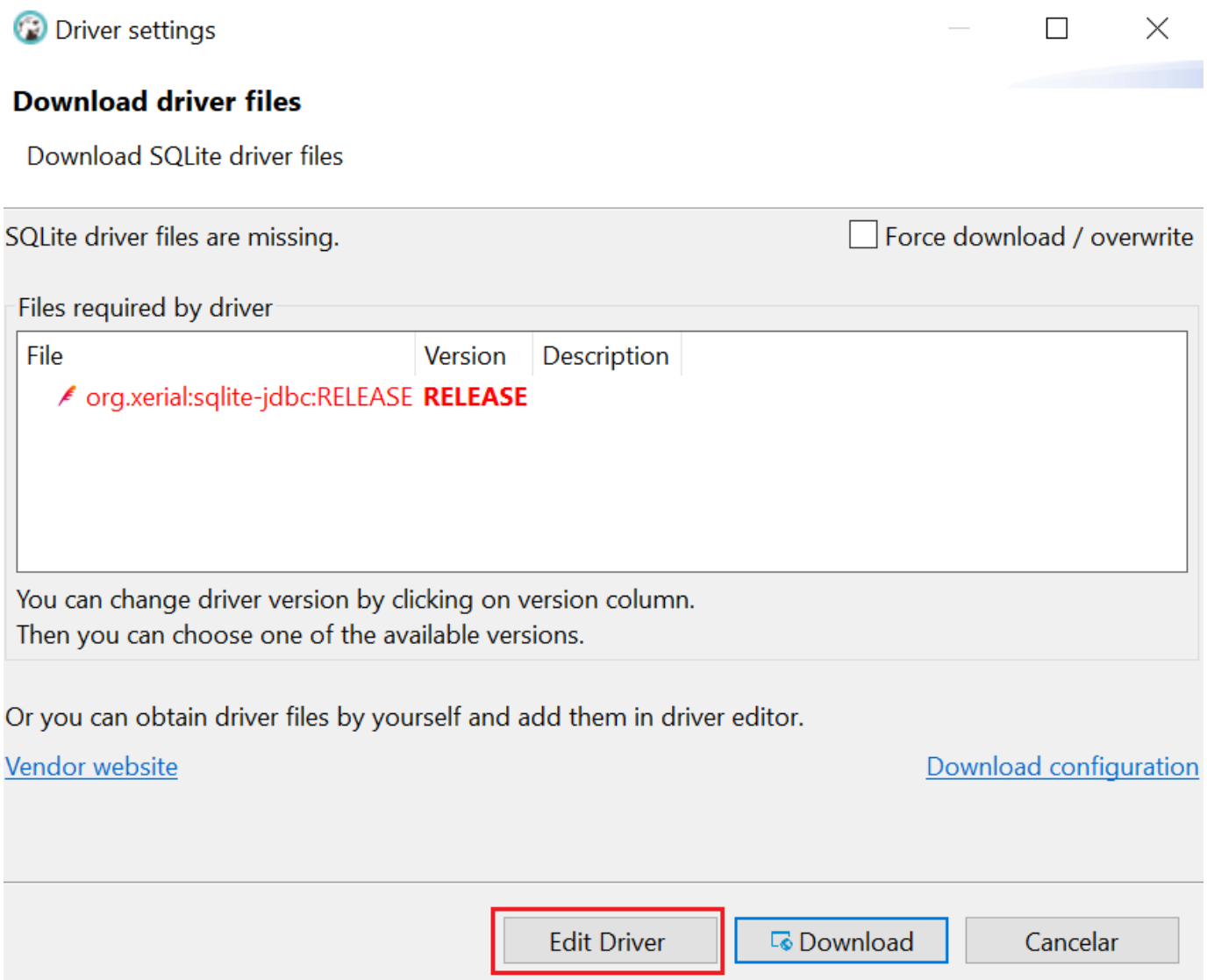
3. Introduce la ruta donde se encuentra la BD y prueba conexión



Haz clic en **"Probar conexión"**. Si todo está correcto, verás un mensaje de éxito.
Si DBeaver necesita un controlador (driver), te lo ofrecerá para descargar automáticamente.



Si la descarga falla, ve a https://github.com/xerial/sqlite-jdbc/releases y descarga el archivo **sqlite-jdbc-3.50.3.0.jar**



5. Finaliza y explora

Haz clic en **"Finalizar"**. La nueva conexión aparecerá en el panel lateral izquierdo.
Desde allí puedes:

- Ver tablas, vistas, funciones y procedimientos
- Ejecutar sentencias SQL
- Consultar y modificar registros
- Exportar datos en distintos formatos

ref